

SolarTrade - Project Synopsis Document

SolarTrade - Project Synopsis Document

1. Project Title

Team Members

2. Abstract

3. Problem Statement

4. Proposed Solution

5. Project Objectives

6. Core Technologies Used

7. Database Collections Schema

8. Complete System Architecture & Directory Tree

9. Comprehensive API Reference Sheet

10. Complete System Workflow Diagram

11. Advantages & Innovation

12. Real-World Use Cases

13. Future Enhancements

SolarTrade - Project Synopsis Document

1. Project Title

SolarTrade: A Decentralized Peer-to-Peer Green Energy Microgrid Trading, Crowdfund Investment, and Smart Invoicing Platform.

Team Members

Team Member	Role
Aarya	Frontend Lead & Marketplace Developer
Toshal	Backend Lead & Project Integration
Sonia	Installation & User Profile Module
Swapana	Admin Module & Reports
Karamveer	Investment & Documentation

2. Abstract

SolarTrade is a full-stack MERN (MongoDB, Express.js, React.js, Node.js) web application designed to digitize, decentralize, and democratize access to renewable green energy. The platform introduces a unified software ecosystem that connects residential/commercial power consumers and solar microgrid investors under a single digital administration panel.

The system supports three core user roles: * **Customers (Consumers & Neighbors):** Can customize profiles to choose between Household Buy (Residential) or Commercial Buy (Business), verify identity credentials (KYC), trade surplus solar energy peer-to-peer on a localized grid marketplace, submit smart-meter utility connections, set up auto-pay details, manage wallets, view cleanliness metrics, and download formal invoice PDFs. * **Investors:** Can explore utility-scale community solar projects, sign digital agreements in a full-page simulated contract PDF reader, check funding limits, fund active microgrids via card checkouts or platform wallets, and collect simulated monthly/yearly interest payouts. * **Administrators:** Audit user accounts, review and approve/reject government identity KYC documents, manage global energy plans, oversee crowdfund project development status, and monitor system analytics.

The application utilizes a responsive layout stretched to full laptop screens, modern glassmorphic designs, secure JSON Web Token (JWT) authorization, cryptographic checkout signatures, and an integrated Dummy Bank testing sandbox.

3. Problem Statement

Traditional electricity utility grids operate under highly centralized infrastructure. Households that install rooftop solar panels have no secure, digital mechanism to sell their clean energy surplus directly to nearby consumers. Instead, they must feed it back to municipal companies at low feed-in tariffs, while buyers pay high prices for conventional utility power.

Simultaneously, retail investors interested in renewable energy projects have limited opportunities to pool capital. Crowdfunding systems lack clear legal contract binding, digital signature tracing, and auto-release maturity payouts. Verification pipelines for customer identities (KYC), billing management, and smart-meter auto-pay structures are manually audited and highly fragmented, which delays transaction settlements.

4. Proposed Solution

SolarTrade provides a centralized digital gateway to address microgrid trading and clean energy crowdfund investments:

- * **Decentralized P2P Marketplace:** Enables consumers to list surplus energy and filters listings dynamically for target categories (Household vs Commercial buying).
- * **Enforced Consumer KYC Banners:** Blocks all marketplace transactions for customers until their government IDs are reviewed and approved, while exempting Admin accounts.
- * **Simulated PDF Contract Viewer:** Replaces small inline scrollbars with full-page A4 contract views featuring official letterheads, watermark backing, and signature lines.
- * **Simulated Dummy Bank Sandbox:** Checks card formats and triggers test declines (insufficient funds, reported lost, incorrect CVV) inside the backend.
- * **Smart Auto-Pay Billing & Invoices:** Simulates hourly smart-meter consumptions, updates auto-pay settings, and exports invoice pop-outs to print-to-PDF downloads.

5. Project Objectives

1. **Decentralize Clean Power Trading:** Create an open marketplace for peer-to-peer solar energy exchange.
 2. **Facilitate Microgrid Crowdfunding:** Allow retail investors to fund community solar projects with strict limit checks (₹10,000 to ₹500,000).
 3. **Secure Transactions:** Implement HMAC cryptographic checkout signatures and mock card authorization switches.
 4. **Digitize Identity Verification:** Build KYC review dashboards for administrators.
 5. **Automate Billing:** Deliver smart auto-pay invoicing and printable PDF billing receipts.
-

6. Core Technologies Used

- **Frontend Stack:** React.js (v18), Vite, Tailwind CSS, ChartJS, React-Chartjs-2, Framer Motion, Axios, React Context API.

- **Backend Stack:** Node.js, Express.js, JWT Cookie Parser, Crypto (SHA256 HMAC).
 - **Database:** MongoDB Atlas, Mongoose ODM.
 - **Development Tools:** Postman, Git, MongoDB Compass.
-

7. Database Collections Schema

A. Collection: `users`

Tracks core user credentials and roles. * `_id` (ObjectId): Primary key. * `name` (String): Full legal name. * `email` (String): Unique email. * `password` (String): Hashed password. * `role` (String): Enum: `['Customer', 'Investor', 'Admin']`. * `isVerified` (Boolean): Email verification flag. * `verificationToken` (String): Email verification token. * `createdAt` / `updatedAt` (Date).

B. Collection: `profiles`

Stores profile parameters, wallets, and custom flags. * `user` (ObjectId): Reference to `User`. * `balance` (Number): Current wallet balance (defaults to `0`). * `rewardPoints` (Number): Accrued loyalty points (defaults to `0`). * `badges` (Array): Enum list: `['Green Investor', 'Carbon Saver']`. * `customerType` (String): Preference selector: `['Household', 'Commercial', 'Unconfigured']`. * `verifiedLocation` (String): Checked city boundary. * `referralCode` (String): Custom share code.

C. Collection: `kycs`

Handles identity checks. * `user` (ObjectId): Reference to `User`. * `fullName` (String): Full name on ID. * `dob` (Date): Date of birth. * `panNumber` (String): PAN card pattern. * `aadhaarNumber` (String): 12-digit Aadhaar number. * `bankName` / `accountHolderName` / `accountNumber` / `ifscCode` (String). * `status` (String): Enum: `['Pending', 'Under Review', 'Verified', 'Rejected']`. * `rejectionReason` (String): Explanation if rejected.

D. Collection: `projects`

Stores community solar farms. * `name` (String): Solar field label. * `location` (String): Regional coordinates. * `targetFunding` (Number): Target investment sum. * `fundedAmount` (Number): Current cash raised. * `minimumInvestment` / `maximumInvestment` (Number): Limits per user. * `expectedROI` (Number): Standard annual return. * `status` (String): Enum: `['Funding', 'Operational', 'Completed']`.

E. Collection: `investments`

Tracks locked capitals. * `user` (ObjectId): Reference to `User`. * `project` (ObjectId): Reference to `Project`. * `amount` (Number): Capital value. * `roi` (Number): Locked rate. * `durationMonths` (Number): Lock duration: `[1, 6, 12, 36, 60]`. * `startDate` / `maturityDate` (Date). * `status` (String): Enum: `['Active', 'Released']`.

F. Collection: `investoragreements`

Stores signed legal PDF metadata. * `user` / `project` (ObjectId). * `amount` (Number): Principal value. * `signature` (String): Typed legal signature. * `signedAt` (Date): Sign timestamp. * `agreementText` (String):

Full legal clauses text.

G. Collection: bills

Tracks connection power invoices. * customer (ObjectId): Reference to User . * unitsConsumed (Number): kWh usage. * totalAmount (Number): Total cost. * status (String): Enum: ['Paid', 'Unpaid'] . * dueDate / billDate (Date).

H. Collection: autopaysettings

Configures billing auto-pay. * user (ObjectId): Reference to User . * isActive (Boolean): Toggle status. * paymentMethod (String): Enum: ['UPI', 'Debit Card', 'Net Banking'] . * maxLimit (Number): Maximum transaction cost limit.

I. Collection: energytrades

Tracks P2P transactions. * seller / buyer (ObjectId): References to User . * unitsKwh (Number): Energy volume. * pricePerUnit (Number): Rate per kWh. * totalAmount (Number): Total cost.

8. Complete System Architecture & Directory Tree

```
SolarTrade/
├── backend/
│   ├── config/db.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── billingController.js
│   │   ├── dashboardController.js
│   │   ├── installationController.js
│   │   ├── investmentController.js
│   │   ├── kycController.js
│   │   ├── marketplaceController.js
│   │   ├── notificationController.js
│   │   └── userController.js
│   ├── middleware/
│   │   ├── auth.js
│   │   └── errorHandler.js
│   ├── models/ (User.js, Profile.js, Project.js, Investment.js, KYC.js, Bill.js, ...)
│   ├── routes/ (authRoutes.js, billingRoutes.js, investmentRoutes.js, kycRoutes.js, ...)
│   ├── seed/seed.js
│   ├── utils/asyncHandler.js
│   ├── .env
│   ├── server.js
│   └── package.json
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/ (Loader.jsx, Modal.jsx, Navbar.jsx, Sidebar.jsx)
│   │   ├── context/AuthContext.jsx
│   │   └── pages/ (Home.jsx, Login.jsx, Register.jsx, CustomerDashboard.jsx, CustomerWallet.jsx,
│   │   CustomerMetrics.jsx, ProjectsExplorer.jsx, Billing.jsx, Marketplace.jsx, AdminDashboard.jsx,
│   │   AdminUsers.jsx, ...)
│   │   ├── services/api.js
│   │   ├── App.jsx
│   │   ├── index.css
│   │   └── main.jsx
│   ├── package.json
│   └── vite.config.js
```

9. Comprehensive API Reference Sheet

A. Authentication Module (/api/auth)

- **POST /api/auth/register (Public)**
 - *Payload:* { name, email, password, role }
 - *Response (201):* { success: true, message: 'User registered.' }
- **POST /api/auth/login (Public)**
 - *Payload:* { email, password }
 - *Response (200):* { success: true, token, user } (Sets cookie)
- **POST /api/auth/logout (Public)**
 - *Payload:* None.
 - *Response (200):* { success: true, message: 'Logged out.' }
- **POST /api/auth/forgot-password (Public)**
 - *Payload:* { email }
 - *Response (200):* { success: true, message: 'Reset email dispatched.' }
- **POST /api/auth/reset-password/:token (Public)**
 - *Payload:* { password }
 - *Response (200):* { success: true, message: 'Password updated.' }
- **GET /api/auth/verify-email/:token (Public)**
 - *Response (200):* HTML Redirect / Verification Success.
- **GET /api/auth/me (Protected)**
 - *Response (200):* { success: true, data: user }
- **PUT /api/auth/change-password (Protected)**
 - *Payload:* { currentPassword, newPassword }
 - *Response (200):* { success: true, message: 'Password changed.' }

B. Billing & Payment Module (/api/billing)

- **GET /api/billing/my-bills (Protected - Customer)**
 - *Response (200):* { success: true, data: bills }
- **POST /api/billing/mock-invoice (Protected - Customer)**
 - *Payload:* { unitsConsumed }
 - *Response (201):* { success: true, data: newBill }
- **GET /api/billing/invoice/:billId/download (Protected - Customer)**
 - *Response (200):* { success: true, data: invoiceDetails }
- **GET /api/billing/autopay/settings (Protected - Customer)**
 - *Response (200):* { success: true, data: autopayRecord }
- **PUT /api/billing/autopay/settings (Protected - Customer)**
 - *Payload:* { isActive, paymentMethod, maxLimit }

- *Response (200)*: { success: true, data: updatedAutopay }
- **POST /api/billing/payments/create-intent (Protected)**
 - *Payload*: { amount, type, referenceId } (types: Deposit, Bill, Investment)
 - *Response (201)*: { success: true, data: { checkoutId, signature } }
- **POST /api/billing/payments/verify-signature (Protected)**
 - *Payload*: { checkoutId, signature, paymentMethod, cardDetails, legalNameSignature, agreementAccepted, durationMonths }
 - *Response (200)*: { success: true, data: { transactionId, status } }

C. Profile & Wallet Module (/api/users)

- **GET /api/users/profile (Protected)**
 - *Response (200)*: { success: true, data: profile }
- **PUT /api/users/profile (Protected)**
 - *Payload*: { verifiedLocation, customerType, referralCode }
 - *Response (200)*: { success: true, data: updatedProfile }
- **POST /api/users/profile/photo (Protected)**
 - *Payload*: Multipart Photo File.
 - *Response (200)*: { success: true, url: photoUrl }
- **GET /api/users/wallet/history (Protected)**
 - *Response (200)*: { success: true, data: { balance, rewardPoints, transactions } }
- **GET /api/users/metrics/analysis (Protected - Customer)**
 - *Response (200)*: { success: true, data: { cleanEnergyShare, utilityShare, carbonOffsetKg } }
- **GET /api/users/rewards (Protected)**
 - *Response (200)*: { success: true, data: { rewardPoints } }
- **POST /api/users/referral (Protected)**
 - *Payload*: { referralCode }
 - *Response (200)*: { success: true, message: 'Referral applied.' }
- **POST /api/users/deposit (Protected - Fallback)**
 - *Payload*: { amount }
 - *Response (200)*: { success: true, balance }
- **DELETE /api/users/account (Protected)**
 - *Response (200)*: { success: true, message: 'Account deleted.' }

D. Crowdfund Investment Module (/api/investments)

- **GET /api/investments/projects (Protected)**
 - *Response (200)*: { success: true, data: projects }
- **POST /api/investments/projects (Admin Only)**
 - *Payload*: { name, location, targetFunding, minimumInvestment, maximumInvestment, expectedROI }
 - *Response (201)*: { success: true, data: newProject }

- **GET /api/investments/portfolio (Protected - Investor)**
 - *Response (200):* { success: true, data: portfolioSummary }
- **GET /api/investments/all (Admin Only)**
 - *Response (200):* { success: true, data: investments }
- **POST /api/investments/withdraw/:investmentId (Protected - Investor)**
 - *Response (200):* { success: true, message: 'Withdrawal submitted.' }
- **POST /api/investments/simulate-payouts (Protected - Investor)**
 - *Response (200):* { success: true, message: 'Interest yields processed.' }
- **GET /api/investments/admin/withdrawals (Admin Only)**
 - *Response (200):* { success: true, data: pendingWithdrawals }
- **PUT /api/investments/admin/withdrawals/:id/approve (Admin Only)**
 - *Response (200):* { success: true, message: 'Withdrawal approved.' }

E. Identity Validation Module (/api/kyc)

- **POST /api/kyc/submit (Protected)**
 - *Payload:* Multipart files for profilePhoto, identityProof, addressProof + form metadata (PAN, Account Info).
 - *Response (201):* { success: true, message: 'KYC submitted.' }
- **GET /api/kyc/status (Protected)**
 - *Response (200):* { success: true, data: kycRecord }
- **GET /api/kyc/admin/all (Admin Only)**
 - *Response (200):* { success: true, data: kycSubmissions }
- **PUT /api/kyc/admin/:id/status (Admin Only)**
 - *Payload:* { status, rejectionReason }
 - *Response (200):* { success: true, data: updatedKyc }

F. Localized Marketplace Module (/api/marketplace)

- **POST /api/marketplace/list (Protected - Customer)**
 - *Payload:* { unitsKwh, pricePerUnit, targetCategory }
 - *Response (201):* { success: true, data: listing }
- **GET /api/marketplace/listings (Protected)**
 - *Response (200):* { success: true, data: listings }
- **POST /api/marketplace/buy/:id (Protected - Customer)**
 - *Payload:* { paymentMethod, cardDetails }
 - *Response (200):* { success: true, data: trade }
- **POST /api/marketplace/buy-direct (Protected - Customer)**
 - *Payload:* { projectId, unitsKwh, paymentMethod, cardDetails }
 - *Response (200):* { success: true, data: trade }
- **GET /api/marketplace/trades (Protected)**

- *Response (200):* { success: true, data: trades }
- **GET /api/marketplace/admin/trades (Admin Only)**
 - *Response (200):* { success: true, data: allTrades }

G. System Announcements & Scope Alerts (/api/notifications)

- **GET /api/notifications (Protected)**
 - *Response (200):* { success: true, data: notifications } (Filters scope dynamically)
- **PUT /api/notifications/read/:id (Protected)**
 - *Response (200):* { success: true }
- **DELETE /api/notifications/:id (Protected)**
 - *Response (200):* { success: true }
- **POST /api/notifications/announcement (Admin Only)**
 - *Payload:* { title, message, category }
 - *Response (201):* { success: true, data: announcement }

H. Rooftop Installation Tracker (/api/installations)

- **POST /api/installations/request (Protected - Customer)**
 - *Payload:* { address, roofAreaSqFt, avgMonthlyBill }
 - *Response (201):* { success: true, data: request }
- **GET /api/installations/my-requests (Protected - Customer)**
 - *Response (200):* { success: true, data: requests }
- **GET /api/installations/all (Admin Only)**
 - *Response (200):* { success: true, data: allRequests }
- **PUT /api/installations/:id/status (Protected)**
 - *Payload:* { status, notes }
 - *Response (200):* { success: true }
- **PUT /api/installations/:id/report (Protected)**
 - *Payload:* Multipart Photo File.
 - *Response (200):* { success: true }

I. Support Board Module (/api/support)

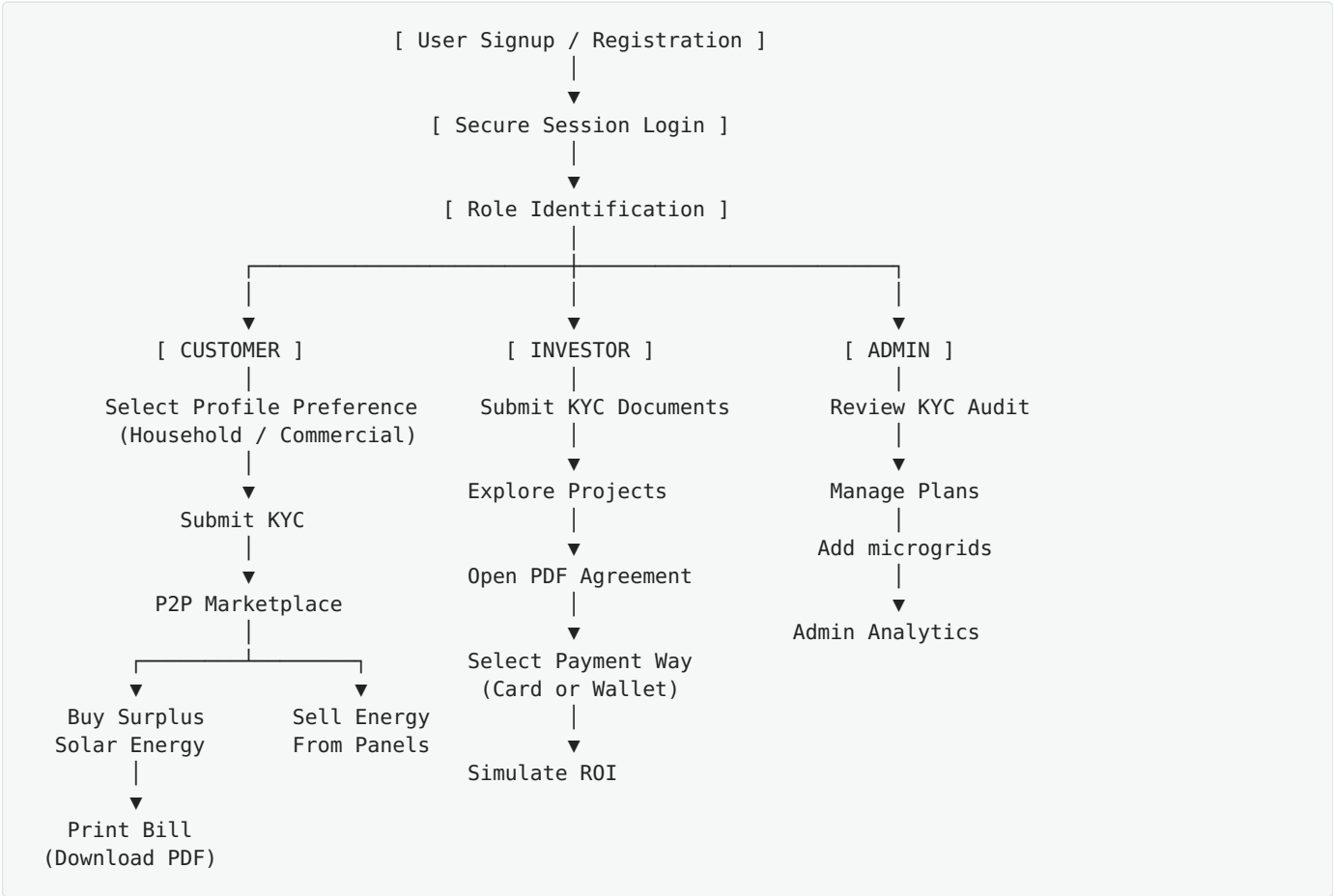
- **POST /api/support/tickets (Protected)**
 - *Payload:* { subject, message, category }
 - *Response (201):* { success: true, data: ticket }
- **GET /api/support/tickets (Protected)**
 - *Response (200):* { success: true, data: tickets }
- **POST /api/support/tickets/:id/reply (Protected)**
 - *Payload:* { message }

- *Response (200):* { success: true }
- **POST /api/support/feedback (Protected)**
 - *Payload:* { rating, comments }
 - *Response (201):* { success: true }
- **GET /api/support/feedback (Admin Only)**
 - *Response (200):* { success: true, data: feedback }

J. System Maintenance Module (/api/maintenance)

- **POST /api/maintenance/request (Protected - Customer)**
 - *Payload:* { projectId, issueType, urgency, description }
 - *Response (201):* { success: true, data: request }
- **GET /api/maintenance/my-requests (Protected - Customer)**
 - *Response (200):* { success: true, data: requests }
- **GET /api/maintenance/all (Protected - Admin)**
 - *Response (200):* { success: true, data: allRequests }
- **PUT /api/maintenance/:id/update (Protected - Admin)**
 - *Payload:* { status, resolutionNotes }
 - *Response (200):* { success: true }

10. Complete System Workflow Diagram



11. Advantages & Innovation

- **P2P Microgrid Arbitrage:** Enables clean energy producers to sell excess power directly to neighbors at customized price levels, bypassing centralized utility grid feed-in tariff limits.
 - **Comprehensive KYC Checks:** Prioritizes secure transacting environments by blocking unverified consumers from checkouts, while exempting administrative accounts.
 - **Simulated PDF Contract Viewer:** Elevates retail investor accessibility with full-page mock A4 agreement layouts, clear watermark indicators, lock rules, and signature verification pads.
 - **Integrated Dummy Bank Sandbox:** Simulates card checkout failures (CVV codes, reported lost cards, insufficient funds limits) on the server, ensuring robust validation handling.
 - **Smart Invoicing System:** Simulates smart-meter power readings and generates functional print-to-PDF downloads.
 - **Glassmorphic Design System:** Provides layout screens optimized for full laptop browser windows.
-

12. Real-World Use Cases

1. **Peer-to-Peer Neighborhood Energy Trading:** Residential homeowners with solar panels list excess energy, and commercial businesses buy it at custom rates.
 2. **Community Solar Infrastructure Funding:** Retail investors pool capitals under legally binding agreements to construct solar farms, receiving automated ROI yield releases.
 3. **Digital Utility Invoice Audits:** Smart-meter setups generate digital invoice receipts. Customers can print or download these invoices as PDF files.
-

13. Future Enhancements

- **IoT Smart Meter Relays:** Real-time energy data synchronization.
- **AI-Powered Arbitrage:** Automated listing price recommendations based on peak load hours.
- **Blockchain Trade Ledgers:** Multi-node contract tracing for P2P energy trades.
- **Native Mobile Application:** Cross-platform React Native integration.